

MSVC Modules Support

Handoff to the Windows Team

Sourcetrail-TS · SOURCETRIL_CXX_MODULES dual build · July 2026

Companion to `post-migration-guide.typ` — read that first.

Scope. The C++20-modules dual build is complete and verified on macOS / Homebrew clang 22.1.8: ~15 first-party modules, 44 module units, ~290 importing TUs, moc coexistence, both build modes green. This document hands the **Windows / MSVC bring-up** to you. It separates what is compiler-neutral design (take as given), what is clang-specific and does not transfer, and what is educated hypothesis about MSVC that you must verify — ranked by risk, with a concrete bring-up plan and a flag/command mapping. Nothing in this document has been executed against MSVC; treat every green box as a contract and every blue box as a test to run.

Contents

1	What transfers unchanged (design contracts)	2
1.1	The dual build itself	2
1.2	The file pattern	2
1.3	The attachment model — keep it, even where MSVC would forgive you	2
1.4	moc coexistence	2
1.5	The playbook and the tooling	2
2	What does not transfer (clang/macOS-specific)	2
3	MSVC-specific landscape (hypotheses to verify)	3
3.1	Toolset and flags	3
3.2	IFC properties you will have to manage	3
3.3	<code>import std</code>	3
3.4	The scanner and stdexec	3
4	Ranked risk register (test in this order)	4
5	Recommended bring-up plan	4

1 What transfers unchanged (design contracts)

These are properties of the **source tree and CMake**, not of clang. Do not redesign them; if MSVC fights one of them, that is a finding to bring back, not a license to fork the pattern.

1.1 The dual build itself

Verified (clang/macOS). `SOURCETRAIL_CXX_MODULES=OFF` (default) must always build and pass the full suite — it is the compatibility guarantee and your safety rail during bring-up. The flag runs through a compiler-support probe (`cmake/SourcetrailCxxModules.cmake`) that **auto-falls back to OFF**; extend its gate for MSVC rather than bypassing it.

1.2 The file pattern

Header + `.cppm` wrapper split; declarations carry `SRCTRL_EXPORT`; headers guard cross-module includes with `#ifndef SRCTRL_MODULE_PURVIEW`; all definitions are `inline` in `.inl` files included at header end. `module`; `/ export module` are never behind `#ifdef` (standard requirement, not a clang one).

1.3 The attachment model — keep it, even where MSVC would forgive you

`SRCTRL_EXPORT` is `export extern "C++"` and every wrapper wraps its textual purview in an `extern "C++" { }` block, attaching **everything** to the global module (`[module.unit]/7`). On clang this was forced by mangling (strong ownership) and by moc. MSVC uses **weak ownership** — module-attached entities mangle the same as classic ones — so several failure modes that forced our hand simply cannot manifest on MSVC.

Risk 0. That is precisely why you must **not** relax the model on Windows: MSVC will happily **accept** mixed attachment states that are ill-formed (`[basic.link]/10`) and that clang rejects. “MSVC builds it” is not evidence of correctness — the macOS CI is the semantic referee. Weak ownership makes MSVC the forgiving platform; keep the source of truth strict.

1.4 moc coexistence

Per-source `SRCTRL_MODULE_BUILD` (CMake `set_property(SOURCE ... APPEND PROPERTY COMPILE_DEFINITIONS ...)`)

the importing TUs; moc-generated TUs and unconverted TUs never see it and keep the classic textual view. This is pure CMake and transfers verbatim. AUTOMOC on Windows produces the same `mocs_compilation.cpp` shape.

1.5 The playbook and the tooling

All twelve playbook rules in `tools/modules-migration/README.md` are standard-derived or pattern-derived except where marked clang-specific. `convertible.py` (analysis, `--apply`, `--index`, `--audit-gmf`) is textual / self-index based and compiler-neutral; use it the same way. The verification protocol (ON: build + full suite + fixed-fixture index equivalence; OFF: build + full suite) is the acceptance bar on Windows too.

2 What does not transfer (clang/macOS-specific)

Skip these; do not port them:

- The Homebrew `std.cppm` INFINITY/NAN patch and the `CMAKE_CXX_STDLIB_MODULES_JSON` repointing in `cmake-toolchains/llvm-clang-macos.cmake` (guarded by an `EXISTS` check on a Homebrew path — inert on Windows, but confirm it stays inert).
- `-fno-modules-reduced-bmi` and the reduced-BMI regression class. MSVC IFCs have no reduced/full split.
- `/usr/bin/ar` vs `llvm-ar` for the std-module archive.
- The clang frontend SEGV that made one TU’s import order significant (alphabetical order is safe here; if MSVC ICEs in an importing TU, permuting imports is still a cheap first experiment — frontends share failure shapes).
- `-wdecls-in-multiple-modules` history (clang diagnostic; removed anyway).

3 MSVC-specific landscape (hypotheses to verify)

3.1 Toolset and flags

Hypothesis — verify on MSVC. Baseline: the design doc’s floor is MSVC 19.36 (VS 17.6), but modules fixes land in nearly every toolset update — use the **newest** toolset available, pin it, and record the exact version in the bring-up notes. Modules effectively require `/permissive-` (default with `/std:c++20+`) and two-phase lookup; our code is already clean under clang’s stricter lookup, but headers that only ever compiled under MSVC-permissive on Windows may surface new errors in module contexts.

Flag / concept mapping for hand-driven experiments (CMake drives all of this automatically via `FILE_SET CXX_MODULES`; you need the raw spellings only to port the smoke tests):

clang (as used in our scripts)	MSVC equivalent
<code>--precompile -x c++-module x.cppm -o x.pcm</code>	<code>cl /std:c++latest /c /interface /TP x.cppm /ifcOutput x.ifc</code>
<code>-fmodule-file=name=x.pcm</code>	<code>/reference name=x.ifc</code>
<code>BMI (.pcm)</code>	<code>IFC (.ifc)</code>
<code>clang-scan-deps (P1689)</code>	<code>cl /scanDependencies (P1689)</code>
<code>-fmodule-output</code>	<code>/ifcOutput</code>
module partitions <code>a:p</code>	same syntax; IFC name <code>a-p.ifc</code>

3.2 IFC properties you will have to manage

Hypothesis — verify on MSVC. IFCs are **toolset-version-locked**: every TU in a build must consume IFCs produced by the same compiler version, and a toolset update invalidates all of them. Consequences: CI caches must key BMI artifacts on the exact `cl.exe` version, and developer machines mid-update produce confusing “cannot open ifc / version mismatch” errors — expect them, script the clean.

Hypothesis — verify on MSVC. Debug info: entities that reach codegen through an IFC have historically had weaker PDB fidelity than textual builds. The dual build is your instrument: any debugging regression can be checked in OFF mode. Not a blocker; note findings.

3.3 `import std`

Hypothesis — verify on MSVC. MSVC is the **best** platform for `import std` (shipped stable with `/std:c++23-era` toolsets; CMake’s `CXX_MODULE_STD` experimental gate covers MSVC). `SOURCETRAIL_CXX_IMPORT_STD=ON` should need **none** of the macOS patching. The two source-level rules still apply: qualify `std::size_t` (bare `::size_t` is not exported) and keep POSIX/C-platform headers textual. Windows adds its own: `<windows.h>` and friends are GMF-only material — never in a purview, and watch `min / max` macros (`NOMINMAX` is already project policy; verify it holds in wrappers).

3.4 The scanner and `stdexec`

Hypothesis — verify on MSVC. Our project-wide `set(CMAKE_CXX_SCAN_FOR_MODULES OFF)` + per-file opt-in exists because `clang-scan-deps` chokes on `stdexec`. MSVC’s `/scanDependencies` may or may not cope — it does not matter: the same four `stdexec` TUs (`TaskBuildIndex.h` consumers, `Schedulers.h`) stay classic on all platforms, and the per-file scan list is compiler-neutral. Do not widen scanning on Windows just because it appears to work; the scan list is part of the design.

4 Ranked risk register (test in this order)

Risk 1. The purview-wide `extern "C++" { }` block. This is the single most MSVC-sensitive construct we use: a linkage-specification containing thousands of declarations, nested linkage-specs, and `export extern "C++"` declarations inside it. clang implements `[module.unit]/7` attachment for it correctly; MSVC's implementation of GM-attachment-via-linkage-spec at this scale is unproven to us. **Smoke-test before anything else** — port `tools/modules-migration/repro-gmf-attachment/` and the two mini shapes from the guide (`export extern "C++"` on every declaration form; the wrapper-block form) to `cl` using the mapping table. If MSVC mishandles attachment here, everything downstream changes — report back before proceeding.

Risk 2. Qt/moc parity. Same AUTOMOC, but Windows Qt is a different binary distribution. The specific thing to verify is the coexistence table from the guide: a converted widget TU and its moc TU must agree on every entity and symbol. A quick `dumpbin /symbols` diff of one widget object file between OFF and ON (they should reference identical decorated names for shared entities under weak ownership + GM attachment) is a cheap strong check.

Risk 3. ICE surface. MSVC's modules frontend historically ICEs on template-heavy purview content (our settings family: `glaze`, `toml++` in GMFs; the Clang-header GMF in `srcctl.cxx` is macOS-only territory but the pattern of 40 MB GMFs is not). Bring modules up **bottom-up in dependency order** — `aidkit`, `srcctl.utility`, `srcctl.file`, ... — one wrapper at a time, not all 44 units at once. CMake builds them in dependency order anyway; the point is to **triage** in that order.

Risk 4. Linker semantics under weak ownership. Our model guarantees one entity, ordinary mangling, inline weak definitions merged across TUs. MSVC COMDAT folding should handle this identically to the classic build — but verify singletons specifically (`MessageQueue::s_instance`, `ColorScheme instance`): exactly one object at runtime in ON mode. A process-wide duplicate singleton is the classic silent failure of ODR-adjacent builds.

Risk 5. Build-system interaction. Use the Ninja generator for bring-up (it is what all our module builds have ever used); the Visual Studio generator's modules support has its own scheduling and should be a later, separate validation if needed. Keep `/STACK:268435456` on the indexer (already in CMake — the AST visitor genuinely needs it on Windows).

5 Recommended bring-up plan

1. **W0 — gate + gauntlet.** Extend the probe in `cmake/SourcetrailCxxModules.cmake` to accept MSVC $\geq$ your pinned toolset; confirm OFF mode is entirely unaffected on Windows (build + full suite). Deliverable: OFF green with the new gate in place.
2. **W1 — smoke the attachment model** (risk 1) with the ported reproducers. Half a day; decides everything. Deliverables: `run-msvc.bat` (or CMake variant) committed next to `run.sh`, and a short findings note.
3. **W2 — modules bottom-up.** `ninja` the ON configure and triage wrapper failures in dependency order (risk 3). Expect the failure **categories** from the guide's war stories — attachment complaints, missing transitive includes — but with MSVC diagnostics. The playbook's reasons sections tell you which category you are in. Deliverable: all 44 module units compile.
4. **W3 — importers + moc.** Build the full ON tree; run the risk-2 and risk-4 checks; then the full acceptance gauntlet: ON build + suite + fixed-fixture index equivalence, OFF build + suite.
5. **W4 — conserve.** Append a "MSVC findings" section to the post-migration guide (same directory): toolset version, deviations found, any new playbook rules. The guide is the living document; the phase log in `context/DESIGN_INDEXER_MODULARIZATION.md` gets the narrative entry.

Escalation: suspected MSVC bugs go to Developer Community with a reduced reproducer — the reduction method in the guide (extract the failing command, inspect which module units textually parsed what, shrink to the minimal module graph) works identically; only the BMI-inspection step changes (`strings` on `.pcm` → the IFC is a structured format; start from `/dlreportTimeSummary` and the module-unit inputs list in the build log instead).

What we will do from the macOS side: nothing lands on `main` that breaks the OFF build anywhere; ON-mode breakage reported from Windows gets a joint triage — bring the exact failing command and the module unit list, per the guide’s diagnosis method.

Pointers.

`post-migration-guide.typ` (same directory — the full findings), `tools/modules-migration/README.md` (living `playbook`), `tools/modules-migration/repro-gmf-attachment/` (reproducers to port), `context/DESIGN_INDEXER_MODULARIZATION.md` (phase log), `cmake/SourcetrailCxxModules.cmake` (the gate to extend). Standard: [module.unit]/7, [basic.link]/10, [dcl.link]. MSVC: “Overview of modules in C++” and “Walkthrough: Build and import STL using modules” on learn.microsoft.com; P1689 dependency format.